

# SIMATS ENGINEERING

## ASSESSMENT TOOL 2: Industry Problem-Based Assignment

COURSE NAME: ARTIFICIAL INTELLIGENCE COURSE CODE: MLA01  
AND EXPERT SYSTEMS

### COURSE OUTCOME COVERED

**CO 5:** Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A

### Industry Problem-Based Assignment

#### Code of Conduct:

*I G.Karthik Reddy-192525047 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

**Signature of the student: G.Karthik Reddy**

| Criterion                                                       | Weightage   | Marks Obtained |
|-----------------------------------------------------------------|-------------|----------------|
| 1. Problem Analysis and Domain Understanding                    | 10%         |                |
| 2. Knowledge Acquisition and Knowledge Representation           | 10%         |                |
| 3. Production Rule / Knowledge Base Design                      | 15%         |                |
| 4. Prolog Implementation and Programming                        | 15%         |                |
| 5. Forward Chaining Implementation and Demonstration            | 10%         |                |
| 6. Backward Chaining Implementation and Demonstration           | 10%         |                |
| 7. Procedural and Non-Procedural Paradigm Analysis              | 10%         |                |
| 8. Unification, Backtracking and Inference Accuracy             | 10%         |                |
| 9. Testing, Results and System Demonstration                    | 5%          |                |
| 10. Documentation, GitHub Repository and Technical Presentation | 5%          |                |
| <b>Total</b>                                                    | <b>100%</b> |                |

Signature of the Faculty

Total Marks Awarded

### ANSWER ANY ONE OF THE FOLLOWING

| S.No. | Industry Domain            | Real-World Problem Statement                                                                                                                                                                                                                                                                                                               |
|-------|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | <b>Automobile Industry</b> | A vehicle service center receives customer complaints such as engine overheating, difficulty starting, abnormal noise, low mileage, and warning-light activation. Develop a Prolog-based expert system that identifies probable vehicle faults and recommends appropriate diagnostic actions using production rules and logical inference. |
| 2     | <b>Healthcare</b>          | A hospital wants to assist preliminary diagnosis by analyzing symptoms such as fever, cough, breathing difficulty, body pain, and fatigue. Develop a rule-based diagnostic system that identifies possible conditions and explains the reasoning behind its conclusions using Prolog.                                                      |
| 3     | <b>Banking</b>             | A bank needs an automated loan pre-screening system that evaluates customer income, credit score, employment status, existing loans, and repayment history. Develop a Prolog-based decision-support system that determines loan eligibility and provides reasons for the decision.                                                         |
| 4     | <b>Cybersecurity</b>       | A Security Operations Center receives events such as repeated failed logins, unusual login locations, privilege escalation, suspicious file access, and abnormal network traffic. Develop an expert system that identifies possible security threats and recommends appropriate actions.                                                   |
| 5     | <b>Manufacturing</b>       | A manufacturing plant experiences unexpected machine failures. Operators provide observations such as abnormal vibration, excessive temperature, unusual noise, pressure changes, and reduced production speed. Develop a machine-fault diagnosis expert system that identifies probable faults and suggests maintenance actions.          |
| 6     | <b>Agriculture</b>         | Farmers observe symptoms such as yellow leaves, spots, wilting, poor growth, and pest infestation. Develop a crop-disease advisory expert system that identifies possible diseases based on crop, soil, weather, and visible symptoms and recommends suitable actions.                                                                     |
| 7     | <b>Telecommunications</b>  | A telecom provider receives complaints about slow internet, frequent call drops, poor signal strength, and network unavailability. Develop an expert troubleshooting system that identifies probable network problems and recommends troubleshooting steps.                                                                                |
| 8     | <b>E-Commerce</b>          | An online retailer wants to recommend products based on customer preferences, purchase history, budget, product category, and previous interactions. Develop a rule-based recommendation system that generates personalized recommendations and explains why each product was recommended.                                                 |

|    |                           |                                                                                                                                                                                                                                                                                                  |
|----|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 9  | <b>Power &amp; Energy</b> | A power distribution company monitors voltage, current, frequency, temperature, and load conditions. Develop an expert system for fault diagnosis that identifies possible overload, voltage instability, equipment failure, or abnormal operating conditions and recommends corrective actions. |
| 10 | <b>Human Resources</b>    | An organization wants to identify suitable training programs for employees based on job role, technical skills, performance, experience, and competency gaps. Develop a rule-based employee training recommendation system that recommends appropriate training and explains the reasoning.      |

### Structure of the Report:

| S.No. | Report Section                              | Expected Content                                                                                                                |
|-------|---------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| 1     | Title, Abstract & Introduction              | Project title, brief abstract, industry background, problem context, and importance of the selected problem.                    |
| 2     | Problem Statement & Objectives              | Clearly define the real-world industry problem, inputs, expected outputs, stakeholders, and specific objectives.                |
| 3     | Domain Analysis & Knowledge Acquisition     | Identify entities, facts, conditions, decisions, and explain how domain knowledge was collected from reliable sources.          |
| 4     | Knowledge Representation & Production Rules | Represent domain knowledge using Prolog facts, predicates, and production rules.                                                |
| 5     | Expert System Architecture                  | Provide and explain the architecture showing the user, knowledge base, inference engine, and output/explanation module.         |
| 6     | Inference Mechanism & Algorithms            | Explain and demonstrate forward chaining, backward chaining, unification, and backtracking with suitable algorithms/pseudocode. |
| 7     | Prolog Implementation                       | Describe the SWI-Prolog implementation, important predicates/rules, queries, and provide the GitHub repository link.            |
| 8     | Test Cases & Results                        | Provide minimum 5 industry-based test cases, queries, expected outputs, actual outputs, and inference traces.                   |
| 9     | Analysis, Limitations & Future Enhancements | Analyze system effectiveness and reasoning accuracy; discuss limitations and propose future improvements.                       |
| 10    | Conclusion & References                     | Summarize the developed expert system, major findings, industry relevance, and provide properly formatted references.           |

## EVALUATION RUBRICS:

| Criterion                                        | Weightage (%) | Excellent                                                                                                                                           | Good                                                                           | Average                                                            | Poor                                                                        |
|--------------------------------------------------|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|--------------------------------------------------------------------|-----------------------------------------------------------------------------|
| 1. Industry Problem Analysis and Understanding   | 10%           | 9–10% – Clearly identifies a relevant real-world industry problem, stakeholders, constraints, inputs, and expected decisions.                       | 7–8% – Good understanding with minor omissions.                                | 5–6% – Basic understanding with limited analysis.                  | 0–4% – Problem is unclear, inappropriate, or poorly analyzed.               |
| 2. Domain Knowledge and Knowledge Acquisition    | 10%           | 9–10% – Acquires comprehensive and reliable domain knowledge and identifies relevant facts, entities, and relationships.                            | 7–8% – Relevant knowledge is acquired with minor gaps.                         | 5–6% – Basic domain knowledge with limited sources.                | 0–4% – Insufficient, unreliable, or inappropriate domain knowledge.         |
| 3. Knowledge Representation and Production Rules | 15%           | 13–15% – Develops a complete, consistent, and well-structured Prolog knowledge base with appropriate facts, predicates, and production rules.       | 10–12% – Well-designed knowledge base with minor omissions or inconsistencies. | 7–9% – Basic knowledge base with several missing or unclear rules. | 0–6% – Incomplete, inconsistent, or inappropriate knowledge representation. |
| 4. Forward and Backward Chaining                 | 15%           | 13–15% – Correctly implements and clearly demonstrates both forward and backward chaining with appropriate industry scenarios and reasoning traces. | 10–12% – Both mechanisms are demonstrated with minor limitations.              | 7–9% – Basic implementation with incomplete demonstration.         | 0–6% – Chaining mechanisms are missing or incorrectly implemented.          |
| 5. Prolog Implementation and Inference           | 15%           | 13–15% – Effectively uses Prolog facts, rules, predicates, unification, backtracking, and inference to generate accurate conclusions.               | 10–12% – Functional implementation with minor technical issues.                | 7–9% – Basic implementation with limited use of Prolog features.   | 0–6% – Implementation is incomplete, incorrect, or non-functional.          |

|                                                         |      |                                                                                                                                              |                                                                   |                                                          |                                                         |
|---------------------------------------------------------|------|----------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|----------------------------------------------------------|---------------------------------------------------------|
| 6. Industry-Based Test Cases and Results                | 10%  | 9–10% – Provides diverse and realistic test cases with accurate queries, outputs, and detailed inference traces.                             | 7–8% – Provides relevant test cases with mostly accurate results. | 5–6% – Limited test cases and basic results.             | 0–4% – Insufficient or inappropriate testing.           |
| 7. Analysis and Interpretation of Results               | 10%  | 9–10% – Provides insightful analysis of reasoning accuracy, consistency, rule coverage, and practical industry usefulness.                   | 7–8% – Good analysis with minor gaps.                             | 5–6% – Basic analysis with limited interpretation.       | 0–4% – Results are poorly analyzed or not interpreted.  |
| 8. Procedural and Non-Procedural Paradigm Understanding | 5%   | 5% – Clearly explains and demonstrates the distinction between procedural and declarative/non-procedural approaches in the developed system. | 4% – Good conceptual distinction with minor gaps.                 | 2–3% – Basic understanding but limited application.      | 0–1% – Incorrect or missing explanation.                |
| 9. Documentation, GitHub and Technical Presentation     | 5%   | 5% – Complete, well-organized report and GitHub repository with clear code, documentation, references, and professional presentation.        | 4% – Good documentation and repository with minor issues.         | 2–3% – Basic documentation with some missing components. | 0–1% – Poor documentation or missing GitHub repository. |
| 10. Limitations, Future Enhancements and Conclusion     | 5%   | 5% – Clearly identifies practical limitations and proposes realistic, technically relevant future enhancements.                              | 4% – Identifies relevant limitations and improvements.            | 2–3% – Basic discussion of limitations and future scope. | 0–1% – Discussion is missing or unrealistic.            |
| Total                                                   | 100% |                                                                                                                                              |                                                                   |                                                          |                                                         |

## **1. Title**

### **Prolog-Based Loan Pre-Screening Expert System**

## **2. Abstract**

A bank receives a large number of loan applications and needs to determine whether customers are suitable for loan approval. Manual evaluation can take time and may produce inconsistent decisions. This project develops a rule-based loan pre-screening expert system using Prolog.

The system analyzes customer information such as income, credit score, employment status, existing loans, and repayment history. Based on predefined production rules, it determines whether a customer is eligible or not eligible for a loan and provides reasons for the decision. The system demonstrates knowledge representation, logical inference, forward chaining, backward chaining, unification, and backtracking.

## **3. Industry Problem**

According to the assignment, a bank needs an automated loan pre-screening system that evaluates:

- Customer income
- Credit score
- Employment status
- Existing loans
- Repayment history

The system must determine **loan eligibility** and provide reasons for the decision.

### **Problem**

Manual loan pre-screening can involve:

- Large numbers of applications
- Repeated verification
- Time-consuming decisions
- Inconsistent preliminary assessment

### **Proposed Solution**

A Prolog expert system can represent banking knowledge as **facts and IF-THEN production rules** and automatically infer a preliminary decision.

**Note:** This is an educational pre-screening system, not a replacement for a bank's actual credit-risk, legal, or compliance process.

## **4. Objectives**

The main objectives are:

1. Develop a Prolog-based loan pre-screening system.
2. Represent customer information as facts.
3. Develop production rules for loan eligibility.
4. Implement logical inference.
5. Demonstrate forward chaining.
6. Demonstrate backward chaining.
7. Demonstrate unification and backtracking.
8. Provide reasons for loan decisions.

9. Test the system using multiple customer cases.

These elements align with the report requirements in the given document.

## **5. Domain Analysis**

### **Input**

The system considers:

| <b>Input</b>      | <b>Example</b>    |
|-------------------|-------------------|
| Income            | High / Low        |
| Credit Score      | Good / Poor       |
| Employment        | Stable / Unstable |
| Existing Loans    | Low / High        |
| Repayment History | Good / Poor       |

### **Output**

The system produces:

- **Eligible**
- **Not Eligible**
- **Reason for decision**

### **Basic Flow**

Customer Information



Knowledge Base



Inference Engine



Production Rules



Loan Eligibility



Reason / Recommendation

## **6. Knowledge Representation**

The knowledge base contains **facts and rules**.

### **Facts**

For example:

income(karthik, high).

credit\_score(karthik, good).

employment(karthik, stable).

existing\_loans(karthik, low).

repayment\_history(karthik, good).

These facts describe the customer's financial information.

## **7. Production Rules**

The rules can be expressed in IF–THEN form.

### **Rule 1 – Eligible Customer**

IF

Income is high

AND Credit score is good  
AND Employment is stable  
AND Existing loans are low  
AND Repayment history is good

THEN

Customer is eligible.

### **Rule 2 – Poor Credit**

IF Credit score is poor

THEN Customer is not eligible.

### **Rule 3 – Unstable Employment**

IF Employment is unstable

THEN Customer is not eligible.

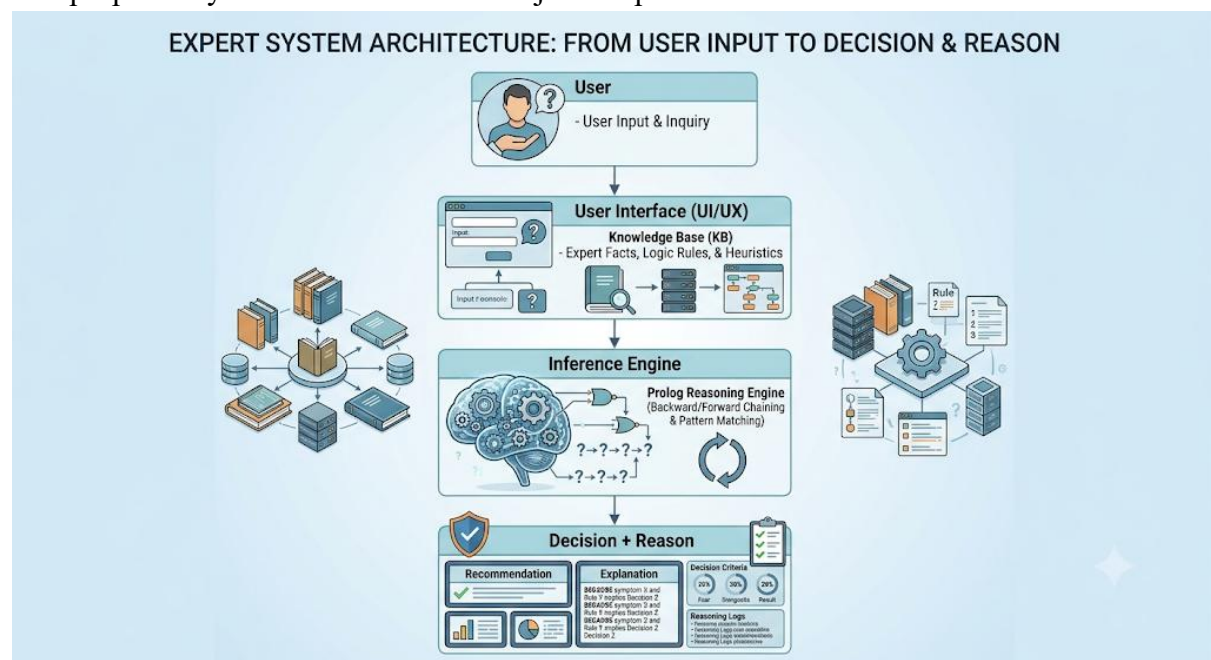
### **Rule 4 – High Existing Loans**

IF Existing loans are high

THEN Customer requires further review.

## **8. Expert System Architecture**

The proposed system consists of four major components:



The report format specifically requires an architecture showing the **user**, **knowledge base**, **inference engine**, and **output/explanation module**.

## **9. SWI-Prolog Implementation**

### **Complete Program**

```
/* =====  
LOAN PRE-SCREENING EXPERT SYSTEM  
===== */  
  
/* ----- Customer Facts ----- */  
  
income(karthik, high).  
credit_score(karthik, good).  
employment(karthik, stable).  
existing_loans(karthik, low).  
repayment_history(karthik, good).  
  
income(ravi, low).  
credit_score(ravi, poor).  
employment(ravi, unstable).  
existing_loans(ravi, high).  
repayment_history(ravi, poor).  
  
income(anu, high).  
credit_score(anu, good).  
employment(anu, stable).  
existing_loans(anu, high).  
repayment_history(anu, good).  
  
/* ----- Production Rules ----- */  
  
/* Rule 1: Loan Eligible */  
loan_status(Person, eligible) :-  
    income(Person, high),  
    credit_score(Person, good),  
    employment(Person, stable),  
    existing_loans(Person, low),  
    repayment_history(Person, good).  
  
/* Rule 2: Poor Credit */  
loan_status(Person, not_eligible) :-  
    credit_score(Person, poor).  
  
/* Rule 3: Unstable Employment */  
loan_status(Person, not_eligible) :-  
    employment(Person, unstable).
```

```
/* Rule 4: High Existing Loans */  
loan_status(Person, review_required) :-  
    existing_loans(Person, high),  
    credit_score(Person, good).
```

```
/* ----- Explanation Rules ----- */
```

```
reason(Person, "Good financial profile") :-  
    loan_status(Person, eligible).
```

```
reason(Person, "Poor credit score") :-  
    credit_score(Person, poor).
```

```
reason(Person, "Unstable employment") :-  
    employment(Person, unstable).
```

```
reason(Person, "High existing loans require review") :-  
    existing_loans(Person, high),  
    credit_score(Person, good).
```

## **10. Running the Program in SWI-Prolog**

Open **SWI-Prolog**, load the program and execute queries.

### **Query 1**

```
?- loan_status(karthik, X).
```

#### **Output**

X = eligible.

#### **Reason**

```
?- reason(karthik, X).
```

Output:

X = 'Good financial profile'.

## **11. Test Case 1 – Eligible Customer**

### **Customer**

#### **Karthik**

Income → High

Credit Score → Good

Employment → Stable

Existing Loans → Low

Repayment History → Good

#### **Query**

```
?- loan_status(karthik, X).
```

#### **Output**

X = eligible.

### **Reasoning**

High Income

+

Good Credit Score

+

Stable Employment

+

Low Existing Loans

+

Good Repayment History

↓

LOAN ELIGIBLE

### **12. Test Case 2 – Poor Credit Score**

#### **Customer**

##### **Ravi**

Income → Low

Credit Score → Poor

Employment → Unstable

Existing Loans → High

Repayment History → Poor

##### **Query**

?- loan\_status(ravi, X).

##### **Output**

X = not\_eligible.

##### **Reason**

Poor Credit Score

↓

Not Eligible

### **13. Test Case 3 – High Existing Loans**

#### **Customer**

##### **Anu**

Income → High

Credit Score → Good

Employment → Stable

Existing Loans → High

Repayment History → Good

##### **Query**

?- loan\_status(anu, X).

##### **Output**

X = review\_required.

##### **Reason**

Good Credit

+  
High Existing Loans  
↓  
Further Review Required

#### **14. Test Cases Summary**

The assignment requires a **minimum of 5 industry-based test cases** with queries, expected outputs, actual outputs, and inference traces.

| Test Case | Customer     | Main Condition         | Expected Result |
|-----------|--------------|------------------------|-----------------|
| TC1       | Karthik      | All conditions good    | Eligible        |
| TC2       | Ravi         | Poor credit            | Not Eligible    |
| TC3       | Anu          | High existing loans    | Review Required |
| TC4       | New Customer | Unstable employment    | Not Eligible    |
| TC5       | New Customer | Good financial profile | Eligible        |

For TC4 and TC5, additional facts can be added to the Prolog program before testing.

#### **15. Forward Chaining**

Forward chaining starts with **known facts** and applies rules to derive new conclusions.

##### **Example**

Initial facts:

High Income

Good Credit

Stable Employment

Low Existing Loans

Good Repayment

Apply:

IF all conditions are satisfied

THEN Eligible

Therefore:

Customer → Eligible

##### **Forward Chaining Flow**

FACTS

↓

MATCH RULE

↓

APPLY RULE

↓

NEW CONCLUSION

↓

LOAN DECISION

The assignment specifically requires demonstration of forward and backward chaining.

## **16. Backward Chaining**

Backward chaining starts with a **goal** and works backward to determine whether the required facts exist.

### **Goal**

?- loan\_status(karthik, eligible).

Prolog checks the rule:

loan\_status(Person, eligible) :-  
    income(Person, high),  
    credit\_score(Person, good),  
    employment(Person, stable),  
    existing\_loans(Person, low),  
    repayment\_history(Person, good).

It then checks:

income(karthik, high)           → TRUE  
credit\_score(karthik, good)     → TRUE  
employment(karthik, stable)    → TRUE  
existing\_loans(karthik, low)    → TRUE  
repayment\_history(karthik, good) → TRUE

Therefore:

true.

## **17. Unification**

Unification allows Prolog to match variables with values.

Consider:

loan\_status(Person, eligible).

For Karthik:

Person = karthik

So Prolog searches for:

loan\_status(karthik, eligible).

This is an important part of Prolog's logical inference process. The assignment rubric explicitly evaluates the use of **unification, backtracking, and inference accuracy**.

## **18. Backtracking**

Prolog automatically tries alternative rules when a rule fails.

For example:

?- loan\_status(ravi, X).

Prolog checks the eligible rule first. If its conditions fail, it backtracks and checks another rule.

It can then find:

X = not\_eligible.

Thus, backtracking allows Prolog to explore different possible solutions.

## **19. Procedural vs Non-Procedural Approach**

| <b>Feature</b>           | <b>Procedural Approach</b>   | <b>Prolog / Non-Procedural Approach</b> |
|--------------------------|------------------------------|-----------------------------------------|
| Focus                    | How to solve                 | What is true                            |
| Programming              | Step-by-step instructions    | Facts and rules                         |
| Control                  | Programmer controls sequence | Prolog inference engine                 |
| Reasoning                | Explicit                     | Automatic                               |
| Example                  | C / Java                     | Prolog                                  |
| Knowledge Representation | Less direct                  | Natural                                 |
| Backtracking             | Usually programmed manually  | Built-in                                |

The assignment specifically asks for understanding and demonstration of the distinction between **procedural and declarative/non-procedural approaches**.

## **20. Advantages**

### **1. Fast Pre-Screening**

The system can quickly check predefined eligibility conditions.

### **2. Consistent Decisions**

The same rules are applied to every customer.

### **3. Explainable Results**

The system can provide the reason behind a decision.

### **4. Easy Knowledge Representation**

Banking rules can be represented using Prolog facts and predicates.

### **5. Automatic Reasoning**

Prolog performs unification, inference and backtracking automatically.

## **21. Limitations**

1. The system uses predefined rules and cannot automatically discover new lending patterns.
2. Real banking decisions require many more factors.
3. Credit regulations and bank policies can change.
4. The simple rule set does not represent the full complexity of real credit scoring.
5. The system should be treated as a **pre-screening aid**, not an autonomous loan approval system.

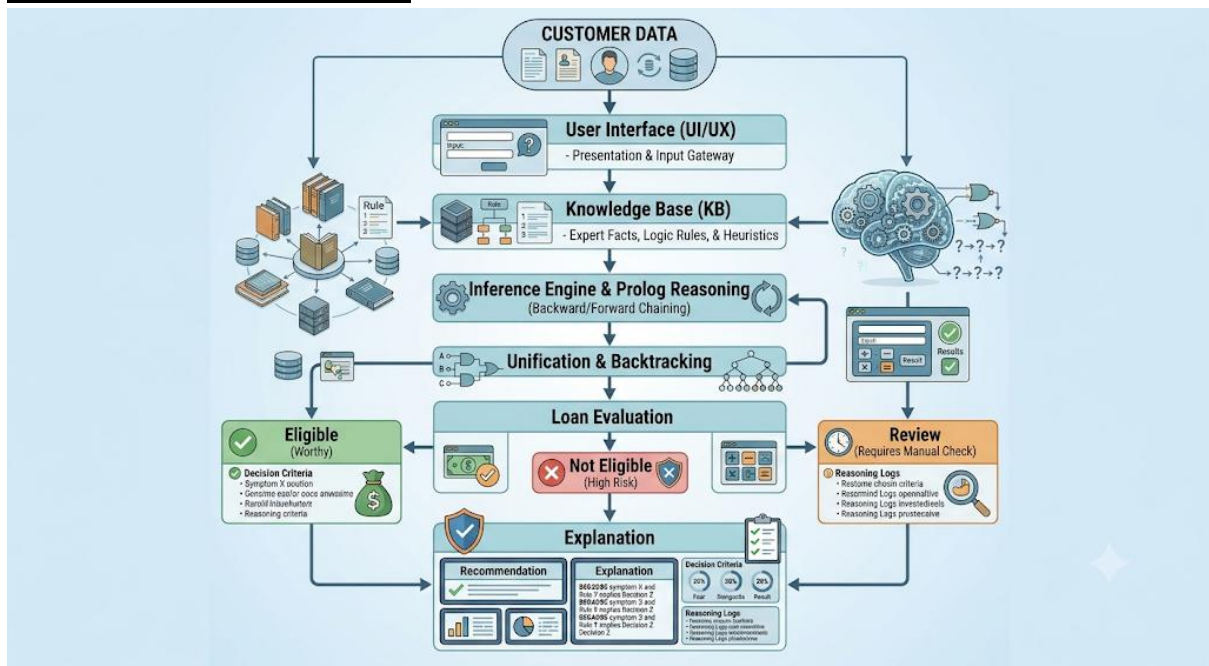
## **22. Future Enhancements**

The system can be improved by adding:

- Machine-learning-based credit-risk prediction.
- Larger customer databases.
- More financial parameters.
- Explainable decision reports.
- Web-based user interface.
- Database integration.
- Real-time credit-score verification.

- Role-based access and security.
- Continuous updating of banking rules.

### 23. Overall System Workflow



### 24. Conclusion

The **Prolog-Based Loan Pre-Screening Expert System** provides a simple and explainable approach for preliminary loan evaluation. Customer information is represented using **facts**, while banking policies are represented using **production rules**.

Using **SWI-Prolog**, the system demonstrates:

- Knowledge representation
- Production rules
- Logical inference
- Forward chaining
- Backward chaining
- Unification
- Backtracking
- Decision explanation

The system can therefore serve as an educational demonstration of how **rule-based expert systems can support banking decisions**. It follows the report structure specified in the assignment, including domain analysis, knowledge representation, architecture, inference mechanisms, implementation, testing, analysis, limitations, future enhancements, and conclusion.

#### Recommended title for submission:

**“Development of a Prolog-Based Loan Pre-Screening Expert System Using Production Rules and Logical Inference”**